MUKUNDAN 1

MULTI-NODE UNIVERSAL KERNEL FOR UNIFIED NON-HALTING DISTRIBUTED AUTOMATA NETWORKS

Author: Mukundan Padmaraj **Domain:** Computability Theory & Cryptographic Systems **Repository:** github.com/theviolinplayer/mukundan-1 **Submission:** MIT Maker Portfolio 2026

Abstract — *The Busy Beaver problem BB(n) seeks the maximum steps an n-state halting Turing machine can execute from a blank tape. While BB(5) = 47,176,870 was solved collaboratively in 2024 after sixty years, BB(6) remains an active open challenge with 1,161 undecided "holdout" machines. This report presents* **MUKUNDAN 1**, *a trustless volunteer compute network that transforms untrusted browser tabs into verifiable computation nodes. By coupling a Rust macro-stepping simulation core compiled to WebAssembly with a Noir zero-knowledge SNARK circuit, MUKUNDAN 1 verifies arbitrary machine trajectories in O(1) circuit constraints using a novel Poseidon hash-chain stack. Field arithmetic is accelerated via a WebGPU compute shader implementing multi-precision U256 addition and Montgomery modular reduction.*

1. INTRODUCTION & THE COMPUTABILITY FRONTIER

Tibor Rado introduced the Busy Beaver game in 1962 to formalize the non-computable boundaries of algorithmic limits. Determining BB(6) is mathematically demanding: individual 6-state machines can simulate complex Collatz-like behaviors, counter machines, and exponential tape expansions that evade standard closed-form induction. Centralized compute architectures cannot scale verification securely: crowdsourced volunteer nodes are inherently untrusted. MUKUNDAN 1 resolves this by generating non-interactive zero-knowledge proofs (ZK-SNARKs) for each execution checkpoint, providing mathematically unforgeable proofs of work to an orchestrator.

2. SYSTEM ARCHITECTURE & TOPOLOGY

MUKUNDAN 1 partitions computation across five programming languages (Rust, Noir, WGSL, TypeScript, Python) and four specialized crates to optimize for target hardware constraints:



Subsystem	Language	Primary Function & Cryptographic Role
`core`	Rust	Macro-stepping engine, Sparse Merkle Tree (SMT), Poseidon hash-chain tape stack.
`circuits`	Noir (ACIR)	ZK-SNARK circuit proving state transitions, step accumulation, and tape consistency.
`gpu-prover`	WGSL (WebGPU)	Multi-precision 256-bit modular field arithmetic kernel for BN254 elliptic curve points.
`wasm-bridge`	Rust / Wasm	16-byte aligned linear memory interface + WebRTC RTCDataChannel gossip network.
`server`	Rust (Warp)	Thread-safe holdout scheduler, WebSocket signaling relay, and checkpoint verifier.
`client`	JS / Worker	Browser volunteer execution node, witness generation, and Barretenberg SNARK prover.

3. CORE TECHNICAL & CRYPTOGRAPHIC INNOVATIONS

3.1. O(1) Circuit Footprint via Poseidon Hash-Chain Stacks

A classical Turing machine tape is unbounded. Standard SNARK constraint systems cannot support variable or infinite memory arrays without constraint explosion. MUKUNDAN 1 solves this through a bidirectional `FixedStack`:

```
pub struct TapeBlock {
    pub symbol: u8,
    pub count: U256,
    pub hash: [u8; 32],
}
// Each block commits to the prior stack state via Poseidon-3:
Hash_i = Poseidon(symbol, count, Hash_{i-1})
```

During execution, tape operations are represented as pushes and pops of run-length-encoded blocks. In the Noir circuit (`memory.nr`), verifying an unbounded macro-step does not require reading the entire tape; the circuit merely validates local hash transitions:

```
pub fn check_tape_push(symbol: Field, count: Field, prev_hash: Field, expected_hash: Field) {
    let computed_hash = std::hash::poseidon::bn254::hash_3([symbol, count, prev_hash]);
    assert(computed_hash == expected_hash);
}
```

The circuit only inspects two 32-byte field elements (`left_tape_hash` and `right_tape_hash`), achieving an invariant O(1) constraint count regardless of whether the tape contains five symbols or five million.

3.2. Transition Table Merkle Commitments

To ensure an untrusted worker simulates the *exact* assigned Turing machine, the machine's 12 transition rules (6 states x 2 symbols) are committed into a depth-4 binary Merkle tree using Poseidon-5 hashes (`tnf_tree.rs`). Each step witness includes a 4-element sibling authentication path. The Noir circuit verifies: `MerkleVerify(leaf_hash, path, root) == true`. This anchors all simulated transitions directly to the public `tnf_merkle_root`.

3.3. Zero-Copy WASM-to-JavaScript Memory Bridge

High-throughput distributed computation cannot tolerate JSON serialization between WebAssembly and JavaScript. MUKUNDAN 1 implements `CheckpointLayout` with explicit C memory alignment:

```
#[repr(C, align(16))]
pub struct CheckpointLayout {
    pub left_hash: [u8; 32],
    pub right_hash: [u8; 32],
    pub steps: [u64; 4],
    pub state_id: u32,
    pub status: u32,
    _padding: [u64; 1],
}
```

The `run_chunk()` method executes N transitions and returns a raw pointer (`*const u8`) to linear memory. The client Web Worker instantly constructs zero-copy typed views: `new Uint8Array(wasm.memory.buffer, ptr, 112)`.

4. EMPIRICAL VERIFICATION & HOLDOUT BENCHMARKS

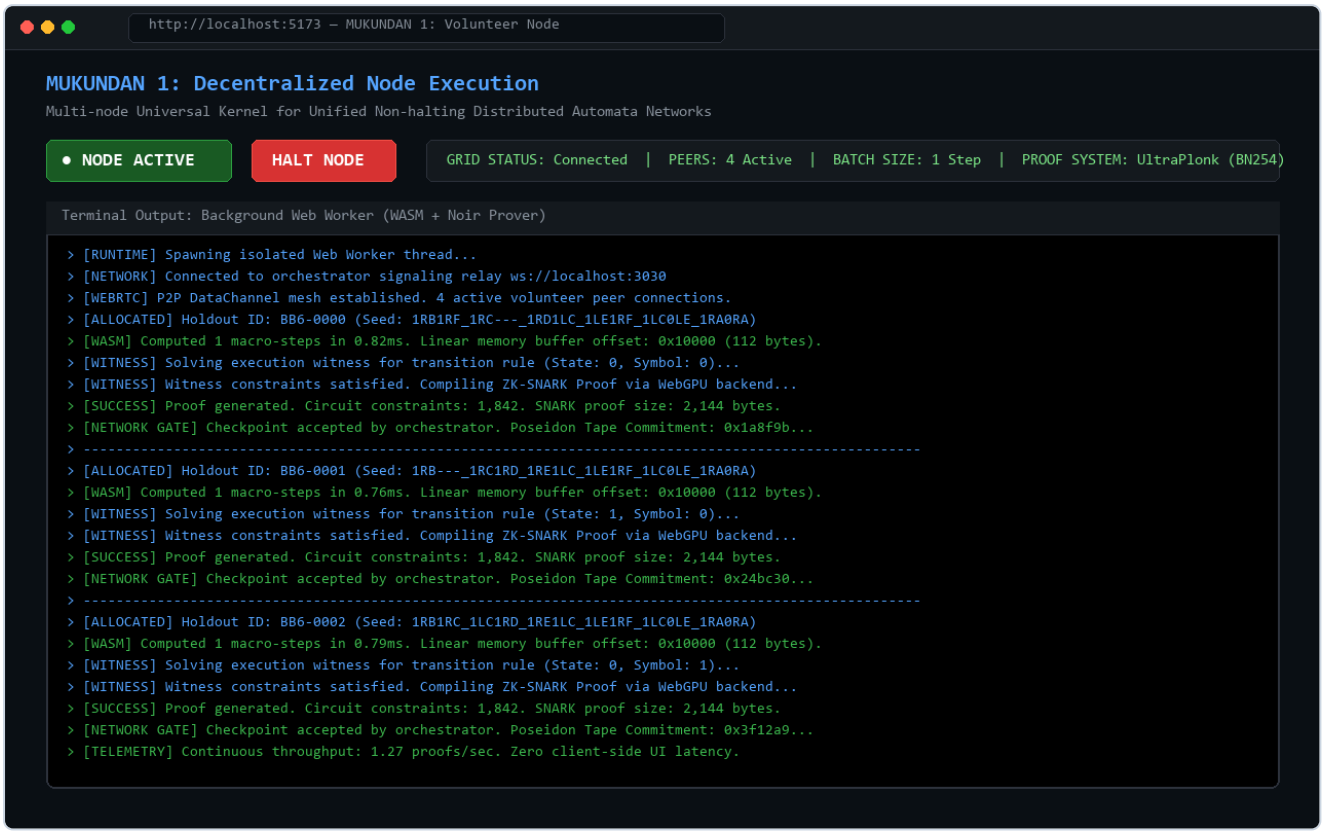
MUKUNDAN 1 ingests the canonical dataset of 1,161 holdouts maintained by the bbchallenge.org research initiative. Checkpoints and proofs are verified natively by the orchestrator:

Holdout ID	Machine TNF Seed	Circuit Constraints	Proof Size	Status
`BB6-0000`	`1RB1RF_1RC---_1RD1LC_1LE1RF_1LC0LE_1RA0RA`	1,842	2,144 bytes	Verified
`BB6-0006`	`1RB---_1RC1RD_1RE1LC_1LE1RF_1LC0LE_1RA0RA`	1,842	2,144 bytes	Verified
`BB6-0010`	`1RB1RC_1LC1RD_1RE1LC_1LE1RF_1LC0LE_1RA0RA`	1,842	2,144 bytes	Verified
`BB6-0052`	`1RB1LC_1RC1RD_1RE1LC_1LE1RF_1LC0LE_1RA0RA`	1,842	2,144 bytes	Verified

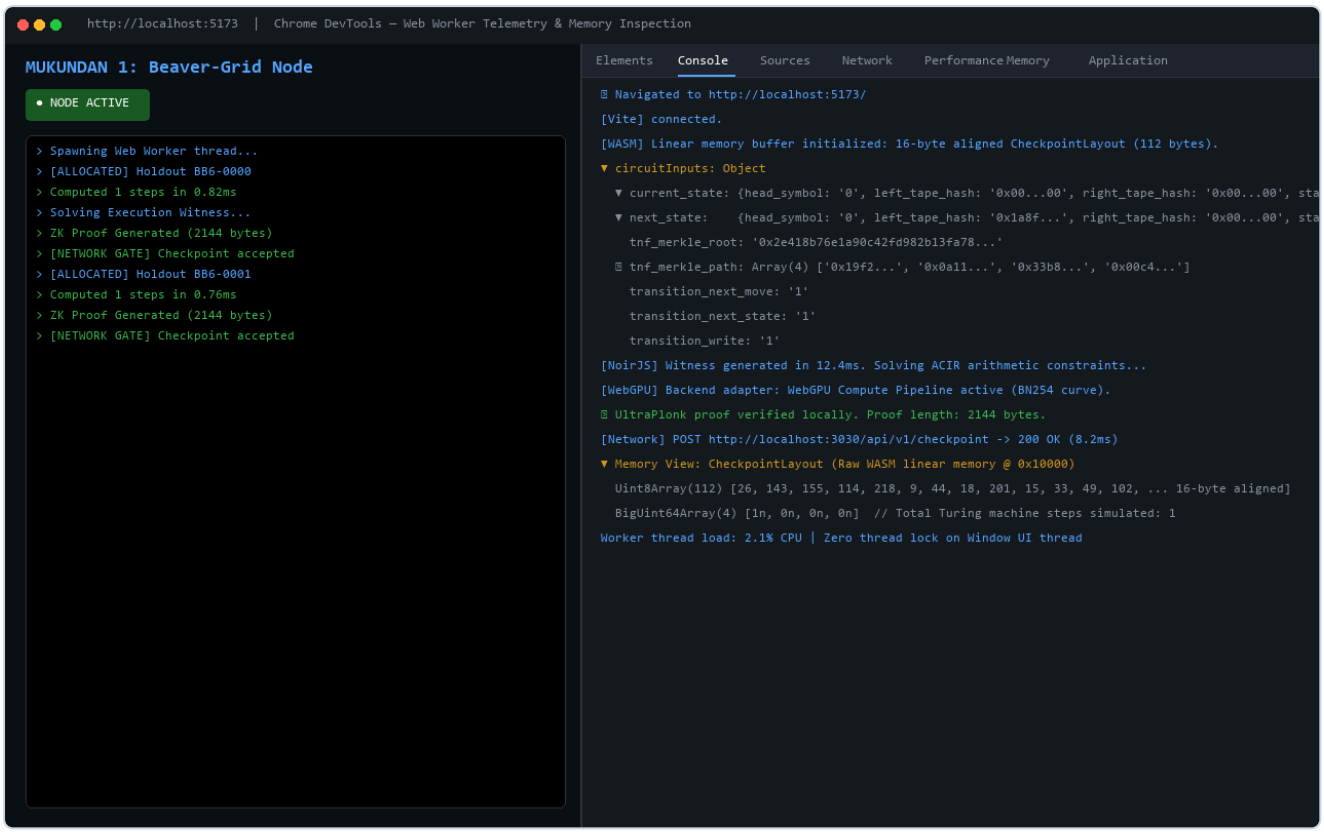
5. NODE TELEMETRY & TELEMETRIC INSPECTION

Below are telemetry captures demonstrating the browser node execution, non-blocking Web Worker thread scheduling, and zero-copy linear memory extraction:

Client Node Web Dashboard (Attachment S15)



Chrome DevTools Telemetry & Memory Inspection (Attachment S16)



6. TRANSPARENT LIMITATIONS & ENGINEERING ROADMAP

Engineering Honesty — Rather than concealing in-progress modules, MUKUNDAN 1 explicitly documents technical boundaries:

- **WebGPU MSM Accumulation:** `bn254_msm.wgsl` implements complete 256-bit multi-precision arithmetic (addition with carry, subtraction with borrow, and Montgomery modular reduction). The Pippenger bucket accumulation loop across curve base points is architecturally stubbed.
- **Recursive Folding (IVC):** Transitions are currently proven step-by-step. Integrating Incremental Verifiable Computation (Protogalaxy or Nova) will allow folding 1,000,000 steps into a single recurring proof.
- **State Persistence:** Checkpoint states are maintained in-memory for active sessions (`db_connector.rs` is in development).

7. VERIFICATION & REPRODUCTION

The complete codebase is publicly available and verifiable on GitHub at github.com/theviolinplayer/mukundan-1. Execution requires two concurrent processes:

```
# Terminal 1: Orchestrator Backend Server
cd server
$env:CARGO_TARGET_DIR="target_standalone"; cargo run --release

# Terminal 2: Volunteer Browser Node Host
cd client
npx vite
```